

API Specification Documentation

OpenAPI Specification for Summit Food Marketplace APIs

1. Introduction

This document defines the API specification for the Summit Food Marketplace project before implementation begins. The purpose of this specification is to describe the externally consumed APIs in a clear, formal, and structured manner using OpenAPI/Swagger standards. This ensures that all teams involved in the project understand the required endpoints, request and response formats, validations, and expected behavior before coding starts.

The project includes several APIs exposed through the Backend for Frontend (BFF) and backend services for products, carts, and supplier approvals.

2. Objective

The objective of this Week 2 task is to finalize the API specification before implementation. The API specification acts as the contract between services and ensures consistency, clarity, and accuracy across the project.

This specification includes:

- API endpoints
- HTTP methods
- request bodies
- response formats
- error handling
- status codes
- authentication/security requirements
- example payloads

3. System Overview

The Summit Food Marketplace contains the following major components:

- Frontend applications
- BFF (Backend for Frontend)
- Product Service
- Cart Service
- Approval Service

These services communicate with each other through APIs. The BFF serves as the main frontend-facing API layer, while internal services handle product management, cart logic, and approval processing.

4. Externally Consumed APIs

The following APIs are considered externally consumed and must be specified before implementation:

1. BFF API
2. Product API
3. Cart API
4. Approval API

These APIs are responsible for the following operations:

- retrieving products
- creating products
- managing carts
- approving and rejecting submissions
- returning JSON responses to the frontend

5. BFF API Specification

5.1 Base URL

http://localhost:8080

5.2 Endpoints

GET /api/products

Purpose:

Returns all products.

Request:

- Query parameter: approved (optional boolean)

Response:

- 200 OK with list of product objects
- 400 Bad Request for invalid input

Example response:

```
{  
  "id": 1,  
  "name": "Fresh Apples",  
  "category": "Fruit",  
  "price": 4.5,  
  "description": "Organic red apples",  
  "stock": 100,  
  "supplier_id": 10,  
  "approved": false,
```

```
"status": "pending",  
"rejection_reason": ""  
}
```

POST /api/products

Purpose:

Creates a new product.

Request body:

- Product object

Response:

- 200 OK with created product
- 400 Bad Request

GET /api/carts/{customer_id}

Purpose:

Returns the cart for a specific customer.

Request:

- Path parameter: customer_id

Response:

- 200 OK with cart object
- 404 Not Found if cart does not exist

POST /api/carts

Purpose:

Creates or updates a cart.

Request body:

- Cart object

Response:

- 200 OK with created/updated cart
- 400 Bad Request

GET /api/submissions

Purpose:

Returns all submissions.

Request:

- Query parameter: status (optional)

Response:

- 200 OK with array of submissions
- 400 Bad Request

PUT /api/submissions/{submission_id}/approve

Purpose:

Approves a supplier submission.

Request:

- Path parameter: submission_id

Response:

- 200 OK with updated submission
- 404 Not Found

PUT /api/submissions/{submission_id}/reject**Purpose:**

Rejects a supplier submission.

Request:

- Path parameter: submission_id
- Query parameter: reason

Response:

- 200 OK with updated submission
- 404 Not Found

6. Product API Specification

6.1 Base URL

http://localhost:8001

6.2 Endpoints

GET /products

Purpose:

Returns a list of products from the product service.

Request:

- Query parameter: approved (optional)

Response:

- 200 OK
- 400 Bad Request

POST /products

Purpose:

Creates a new product.

Request body:

```
{  
  "id": 1,  
  "name": "Fresh Apples",  
  "category": "Fruit",  
  "price": 4.5,  
  "description": "Organic red apples",  
  "stock": 100,  
  "supplier_id": 10,  
  "approved": false,
```

```
"status": "pending",  
"rejection_reason": ""  
}
```

Response:

```
{  
  "id": 1,  
  "name": "Fresh Apples",  
  "category": "Fruit",  
  "price": 4.5,  
  "description": "Organic red apples",  
  "stock": 100,  
  "supplier_id": 10,  
  "approved": false,  
  "status": "pending",  
  "rejection_reason": ""  
}
```

GET /products/{product_id}

Purpose:

Fetches a single product by product ID.

Response:

- 200 OK if product exists
- 404 Not Found if product does not exist

PUT /products/{product_id}

Purpose:

Updates an existing product record.

Request body:

- Product object

Response:

- 200 OK
- 404 Not Found

7. Cart API Specification

7.1 Base URL

http://localhost:8002

7.2 Endpoints

GET /carts/{customer_id}

Purpose:

Returns the cart belonging to the specified customer.

Request:

- Path parameter: customer_id

Response:

```
{  
  "customer_id": 12,  
  "items": [  
    { "product_id": 1, "quantity": 2 },  
    { "product_id": 3, "quantity": 1 }  
  ]  
}
```

POST /carts**Purpose:**

Creates or updates a cart.

Request body:

```
{  
  "customer_id": 12,  
  "items": [  
    { "product_id": 1, "quantity": 2 },  
    { "product_id": 3, "quantity": 1 }  
  ]  
}
```

Response:

```
{  
  "customer_id": 12,  
  "items": [  
    { "product_id": 1, "quantity": 2 },  
    { "product_id": 3, "quantity": 1 }  
  ]  
}
```

DELETE /carts/{customer_id}/items/{product_id}

Purpose:

Removes an item from a customer's cart.

Request:

- customer_id
- product_id

Response:

- 200 OK with updated cart
- 404 Not Found if cart is missing

8. Approval API Specification

8.1 Base URL

http://localhost:8003

8.2 Endpoints

GET /submissions

Purpose:

Returns supplier submissions.

Request:

- Query parameter: status (optional)

Response:

```
[
  {
    "id": 1,
    "supplier_id": 25,
    "product_name": "Fresh Apples",
    "category": "Fruit",
    "price": 4.5,
    "description": "Organic red apples",
    "stock": 100,
    "status": "pending",
    "rejection_reason": ""
  }
]
```

POST /submissions

Purpose:

Submits a product for approval.

Request body:

```
{  
  "id": 1,  
  "supplier_id": 25,  
  "product_name": "Fresh Apples",  
  "category": "Fruit",  
  "price": 4.5,  
  "description": "Organic red apples",  
  "stock": 100,  
  "status": "pending",  
  "rejection_reason": ""  
}
```

Response:

```
{  
  "id": 1,  
  "supplier_id": 25,  
  "product_name": "Fresh Apples",  
  "category": "Fruit",  
  "price": 4.5,  
  "description": "Organic red apples",  
  "stock": 100,  
  "status": "pending",
```

```
"rejection_reason": ""  
}
```

PUT /submissions/{submission_id}/approve

Purpose:

Approves submission.

Response:

```
{  
  "id": 1,  
  "supplier_id": 25,  
  "product_name": "Fresh Apples",  
  "category": "Fruit",  
  "price": 4.5,  
  "description": "Organic red apples",  
  "stock": 100,  
  "status": "approved",  
  "rejection_reason": ""  
}
```

PUT /submissions/{submission_id}/reject

Purpose:

Rejects submission and records rejection reason.

Request:

- submission_id
- reason

Response:

```
{  
  "id": 1,  
  "supplier_id": 25,  
  "product_name": "Fresh Apples",  
  "category": "Fruit",  
  "price": 4.5,  
  "description": "Organic red apples",  
  "stock": 100,  
  "status": "rejected",  
  "rejection_reason": "Price too high"  
}
```

9. API Data Models

Product Model

```
{  
  "id": 1,  
  "name": "Fresh Apples",  
  "category": "Fruit",  
  "price": 4.5,  
  "description": "Organic red apples",  
  "stock": 100,  
  "supplier_id": 10,  
  "approved": false,  
  "status": "pending",  
  "rejection_reason": ""  
}
```

Cart Model

```
{  
  "customer_id": 12,  
  "items": [  
    { "product_id": 1, "quantity": 2 }  
  ]  
}
```

Submission Model

```
{  
  "id": 1,  
  "supplier_id": 25,  
  "product_name": "Fresh Apples",  
  "category": "Fruit",  
  "price": 4.5,  
  "description": "Organic red apples",  
  "stock": 100,  
  "status": "pending",  
  "rejection_reason": ""  
}
```

10. Status Codes

The API uses standard HTTP status codes:

- 200 OK – successful request
- 201 Created – new resource created

- 400 Bad Request – invalid request or payload
- 404 Not Found – resource does not exist
- 500 Internal Server Error – server-side failure

11. Authentication and Security

For Week 2, the API specification should include security requirements for all protected endpoints.

Suggested requirements:

- Authentication is required for user and supplier actions
- Access control is enforced for approval operations
- JWT or bearer token-based authentication should be used
- Request validation should be performed before processing

This ensures that only authorized users can access sensitive API functionality.

12. Conclusion

This API specification defines the contract for all externally consumed APIs in the Summit Food Marketplace project before implementation. It includes product APIs, cart APIs, approval APIs, and BFF routes used by the frontend. The specification documents the required endpoints, expected payloads, errors, and response formats, ensuring consistency and reducing ambiguity during implementation.